

Reviewer Pack — Minimal Index of Lie Algebroid Pairs and Circuit Monotone Wi...

Entry 2f3723959e0f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 17:14:40 UTC

Minimal Index of Lie Algebroid Pairs and Circuit Monotone Width

Entry ID: 2f3723959e0f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 17:14:40 UTC

1. Conjecture

Field A (mathematical branch): Lie Algebroids Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For any given Boolean circuit with monotone width w , there exists a pair of Lie algebroids (L_1, L_2) such that the index of their smash product, $\text{Ind}(L_1 \otimes L_2)$, is linearly correlated with the monotone width w , i.e., $\text{Ind}(L_1 \otimes L_2) = \Theta(w)$.

Rationale (proposer's reasoning):

Lie algebroids provide a non-commutative generalization of vector spaces that can potentially capture the structure of Boolean circuits in ways that are not possible with standard linear algebra. The index of a Lie algebroid pair is an invariant that measures the complexity of their interaction, and it may reveal new insights into circuit complexity.

Taxonomy category: Lie-Algebroid-Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 183b94f9bfee2bba

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between $\text{Ind}(L_1 \otimes L_2)$ and w exceeds 0.7 for at least 80% of the seeds, with no seed producing a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_lie_algebroid(n):
        L = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if i != j and random.choice([True, False]):
                    L[i][j] = random.randint(1, 10)
        return L

    def smash_product(L1, L2):
        n = len(L1)
        result = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    result[i][j] += L1[i][k] * L2[k][j]
        return result

    def index(L):
        n = len(L)
        det = 0
        if n == 1:
            return abs(L[0][0])
        for i in range(n):
            minor = [row[:i] + row[i+1:] for row in L[1:]]
            sign = (-1) ** i
            det += sign * L[0][i] * index(minor)
        return abs(det)

    def monotone_width(circuit):
        # Placeholder function to generate a random monotone width
        return random.randint(5, 20)
```

```

n_max = 30
instances_tested = 0
total_index = 0
total_width = 0

for n in range(5, n_max + 1):
    L1 = generate_lie_algebroid(n)
    L2 = generate_lie_algebroid(n)
    product = smash_product(L1, L2)
    ind = index(product)
    width = monotone_width([random.randint(0, 1) for _ in range(n)])

    total_index += ind
    total_width += width
    instances_tested += 1

if instances_tested == 0:
    return {
        "metric_name": "Index of Lie Algebroid Pairs",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

mean_index = total_index / instances_tested
mean_width = total_width / instances_tested
correlation_coefficient = (instances_tested * total_index * total_width - sum(ind * width for
    return {
        "metric_name": "Index of Lie Algebroid Pairs",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient > 0.7,
        "counterexample": "" if correlation_coefficient >= 0.5 else "Correlation coefficient below
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {'seed': {seed}, ...{result}...}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None) /
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results if res["metric_val
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra

```

```

elif any(not res["conjecture_holds"] and res["counterexample"] == "Correlation coefficient below 0.5") and res["first_failing_seed"] != "":
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"] and res["first_failing_seed"] != "")
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_below_0.5\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data_or_other_reasons")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13566
2	propose	ollama_remote	glm4:latest	0	0	12560
3	propose	ollama_remote	glm4:latest	0	0	16158
4	preregistration	ollama_remote	glm4:latest	0	0	10330
5	novelty	ollama_remote	glm4:latest	0	0	17395
6	novelty	ollama_remote	glm4:latest	0	0	8813
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23163
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8687
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8825
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12822
11	judge	ollama_remote	glm4:latest	0	0	11576

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143894 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/2f3723959e0f.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2f3723959e0f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2f3723959e0f.tar.gz` (if generated)